TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE EVALUACIÓN

01

// Unidad I · Fundamentos de Java

PRIMERA EVALUACIÓN

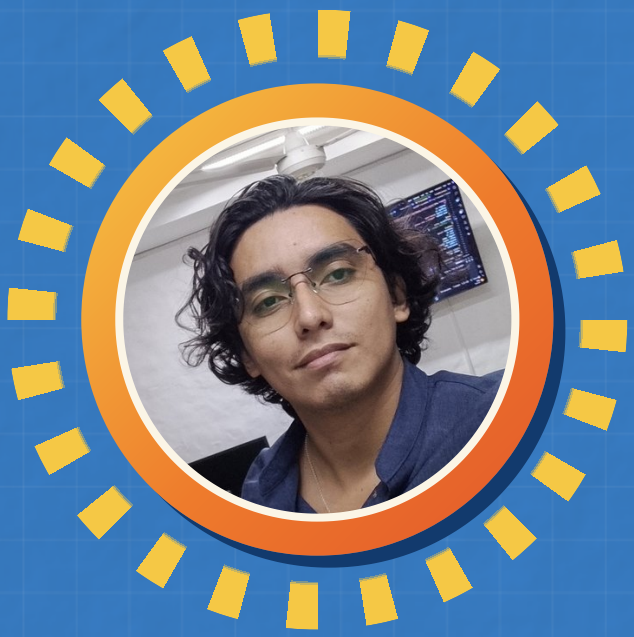
DOS EJERCICIOS, DOS REPOSITORIOS

FECHA LÍMITE

Lunes 02 de noviembre · 8:00 p.m.

EN ESTE DOCUMENTO

01 Información general
02 Ejercicio 1 · Cafetería
03 Ejercicio 2 · Parqueo
04 Cómo entregar
05 Rúbrica de evaluación
06 Antes de enviar



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología Francisco "El Chele" Moreno

Fundamentos de Desarrollo Backend con Java

Octubre 2026

CAPÍTULO 01

1 INFORMACIÓN GENERAL

> Esta primera evaluación tiene **dos ejercicios** que integran todo lo visto en la Unidad I: variables, operadores, estructuras de control y programación orientada a objetos. Cada ejercicio se entrega en **su propio repositorio público** de GitHub.

FECHA LÍMITE DE ENTREGA

LUNES 02 DE NOVIEMBRE DE 2026

Se evalúa el último commit hecho en cada repositorio antes de esta hora.

8:00 P.M.

Aspecto	Detalle
Modalidad	Individual.
Valor	100 puntos: **50** por el Ejercicio 1 y **50** por el Ejercicio 2.
Entrega	Dos repositorios **públicos** en GitHub, uno por ejercicio, y sus enlaces enviados al docente.
Herramientas	JDK 17 o superior (recomendado 25 LTS), VS Code con Extension Pack for Java, Git y una cuenta de GitHub.
Material de apoyo	Lecturas 01, 02 y 03, y los ejemplos `Encapsulamiento.java` y `Herencia.java` del repositorio del curso.

Qué se evalúa

Tema	Material	Dónde lo vas a usar
Variables, tipos y `Scanner`	Lectura 01 · Encuentro 01	Leer opciones, cantidades, placas y minutos del teclado.
Estructuras de control	Lectura 01	Menús con `do-while` y `switch`; validaciones con `if` y `while`.
Operadores y `Math`	Lectura 02	Descuentos, IVA, redondeo y cálculo de horas con `Math.ceil()`.
Clases y objetos	Lectura 03	Ambos ejercicios.
Encapsulamiento	Lectura 03	Ejercicio 1 (eje principal) y Ejercicio 2.
Herencia	Lectura 03	Ejercicio 2 (eje principal).

Reglas generales

- Cada clase va en **su propio archivo** `.java`, con el mismo nombre de la clase.
- El programa debe **compilar y ejecutarse sin errores**. Un programa que no compila no puede recibir los puntos de funcionamiento.

- No necesitas arreglos ni listas: los dos ejercicios se resuelven con lo visto en clase.
- El programa **no debe caerse** con datos inválidos dentro de lo razonable: opciones fuera del menú, cantidades negativas o en cero.
- Puedes consultar las lecturas, la documentación y tus apuntes. El código debe ser tuyo: el docente puede pedirte que expliques cualquier parte.

CAPÍTULO 02 · 50 PUNTOS

2 EJERCICIO 1 · CAFETERÍA

> La cafetería del Centro necesita un **punto de venta en consola**: vender productos, controlar el inventario y ver cuánto se vendió en el día. El eje de este ejercicio es el **encapsulamiento**.

Clases que debes crear



Requisitos de la clase Producto

- Los tres atributos son `private`, con un constructor que los recibe.
- `setPrecio()` solo acepta precios **mayores que 0**; si no, muestra un mensaje y no cambia nada.
- **No debe existir** `setStock()`. El stock solo cambia con `vender(cantidad)`, que devuelve `false` si la cantidad es inválida o no alcanza el stock, y con `reabastecer(cantidad)`, que solo acepta cantidades positivas.

Requisitos de la clase Caja

- `totalVendido` y `numeroVentas` son `private`, con getters y **sin setters**: solo cambian cuando se cobra una venta.
- `cobrar(producto, cantidad)` intenta vender. Si la venta es posible, calcula en este orden e imprime el ticket:

#	Concepto	Regla
1	**Subtotal**	`precio × cantidad`
2	**Descuento**	**10 %** del subtotal si el subtotal es de **C$300 o más**; si no, 0.

#	Concepto	Regla
3	IVA	**15 %** de `subtotal - descuento`, redondeado a 2 decimales.
4	Total	`subtotal - descuento + IVA`, redondeado a 2 decimales.

Para redondear a dos decimales puedes usar `Math.round(valor * 100) / 100.0`. Después de cobrar, la caja suma el total a `totalVendido` y aumenta `numeroVentas`.

Productos iniciales

Producto	Precio	Stock
Café americano	C$45	20
Cappuccino	C$70	15
Sándwich	C$95	10
Galleta de avena	C$30	25

Menú del programa

En `main`, usa un `do-while` con `switch` que repita el menú hasta que el usuario elija **0**:

```
Menú
===== CAFETERÍA CI =====
1. Ver productos
2. Vender
3. Reabastecer
4. Cambiar precio
5. Reporte del día
0. Salir
Elige una opción:
```

Opción	Qué debe hacer
1 · Ver productos	Lista los 4 productos con número, precio y stock. Si un producto tiene **menos de 5 unidades**, muestra `(stock bajo)` a su lado.
2 · Vender	Pide el número del producto y la cantidad, y llama a `caja.cobrar(...)`. Si no hay stock suficiente, avisa sin cobrar.
3 · Reabastecer	Pide producto y cantidad, y llama a `reabastecer()`.
4 · Cambiar precio	Pide producto y nuevo precio, y llama a `setPrecio()`.
5 · Reporte del día	Muestra el número de ventas y el total vendido.
Otra opción	Muestra `Opción inválida` y vuelve a mostrar el menú.

Ejemplo de ticket

Venta de 5 sándwiches. Tu formato puede ser distinto, pero los valores deben coincidir:

```
Salida esperada

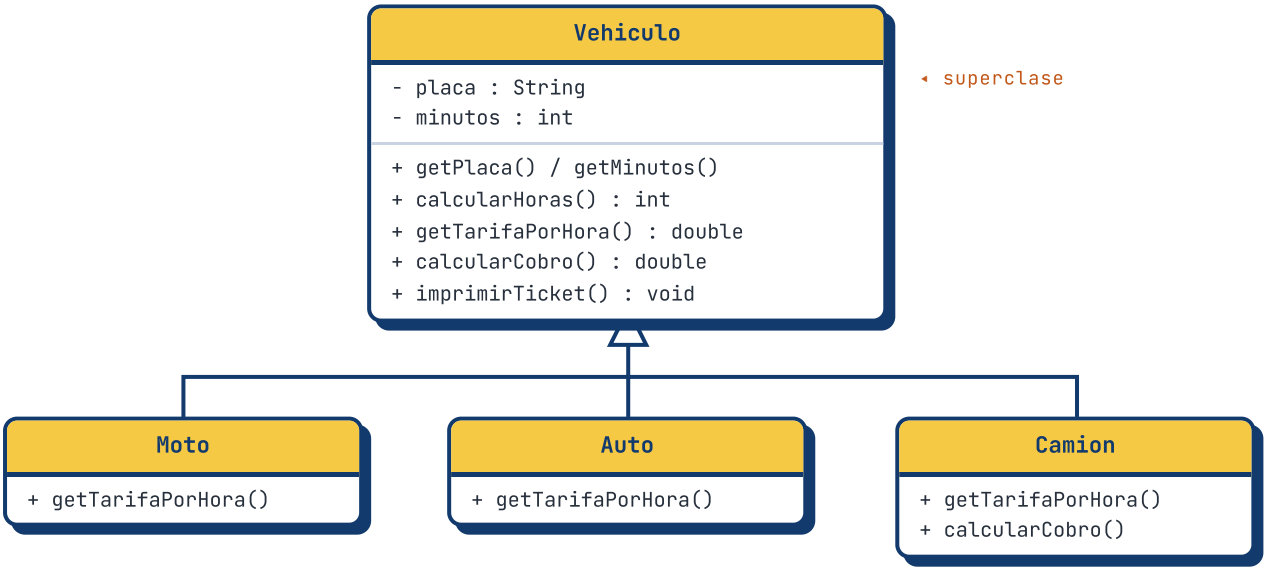
----- TICKET -----
Sándwich x 5
Subtotal:   C$475.0
Descuento:  C$47.5
IVA (15%):  C$64.13
TOTAL:      C$491.63
------------------
```

RETO OPCIONAL

Pide el nombre del cliente al vender e imprímelo en el ticket, junto con un **número de orden** que aumente en cada venta. Ese número también debe ser un atributo `private` de `Caja`.

3 CAPÍTULO 03 · 50 PUNTOS
EJERCICIO 2 · PARQUEO

> El parqueo del Centro cobra distinto según el tipo de vehículo. Necesita un programa que calcule el cobro de cada vehículo que sale y muestre un reporte al cerrar el día. El eje de este ejercicio es la **herencia**.



Requisitos de la clase Vehiculo

- `placa` y `minutos` son `private`, se reciben en el constructor y se leen con getters.
- `calcularHoras()` aplica las reglas del parqueo:

Tiempo estacionado	Horas a cobrar
15 minutos o menos	**0**: los primeros 15 minutos son gratis.

Tiempo estacionado	Horas a cobrar
Más de 15 minutos	Cada hora o fracción se cobra completa: `(int) Math.ceil(minutos / 60.0)`.

- `calcularCobro()` devuelve `calcularHoras() × getTarifaPorHora()`.
- `imprimirTicket()` muestra el tipo de vehículo, la placa, los minutos, las horas cobradas y el total.

Requisitos de las subclases

Clase	Tarifa por hora	Qué sobrescribe
Moto	C$10	`getTarifaPorHora()`
Auto	C$20	`getTarifaPorHora()`
Camion	C$35	`getTarifaPorHora()` y `calcularCobro()`: suma un **recargo fijo de C$50** si cobra al menos una hora. Usa `super.calcularCobro()`.

- Las tres usan `extends Vehiculo`, llaman a `super(placa, minutos)` en su constructor y marcan con `@Override` los métodos que sobrescriben.
- Las subclases **no** repiten `placa` ni `minutos`: los heredan.

RECOMENDADO

Declara `Vehiculo` como `abstract` y `getTarifaPorHora()` como método abstracto. Así nadie puede crear un "vehículo genérico" y cada subclase queda obligada a definir su tarifa.

Flujo del programa

En `main` (clase `Parqueo`), repite con un `do-while` hasta que el usuario responda `n`:

1. Pedir el tipo: `1` Moto, `2` Auto, `3` Camión. Si el tipo no es válido, volver a pedirlo con un `while`.
2. Pedir la placa y los minutos estacionado. Los minutos deben ser **mayores que 0**; si no, volver a pedirlos.
3. Crear el objeto de la subclase correcta, guardarlo en una variable de tipo `Vehiculo` e imprimir su ticket.
4. Acumular el **total recaudado** y contar cuántos vehículos de cada tipo salieron.
5. Preguntar `¿Registrar otro vehículo? (s/n)`.

Al terminar, muestra el **reporte del día**: cantidad de motos, autos y camiones, total recaudado y la **placa del vehículo que pagó más**.

Casos de prueba

Tu programa debe dar exactamente estos cobros:

Tipo	Placa	Minutos	Horas	Cálculo	Cobro
Moto	`M 123-456`	10	0	gratis	**C$0.0**
Auto	`MN 456-789`	61	2	2 × 20	**C$40.0**
Camión	`C 987-654`	130	3	3 × 35 + 50	**C$155.0**
Camión	`C 111-222`	15	0	gratis, sin recargo	**C$0.0**

RETO OPCIONAL

Agrega una cuarta subclase, `Bicicleta`, que siempre cobre C$0 y no cuente para la placa que pagó más. Si tu diseño está bien hecho, solo tendrás que crear la clase y agregar una opción al menú.

4 CAPÍTULO 04 CÓMO ENTREGAR

> Cada ejercicio vive en **su propio repositorio público** de GitHub. Al final envías los dos enlaces al docente.

1 · Nombres de los repositorios

Ejercicio	Nombre exacto del repositorio
Ejercicio 1	`evaluacion01-cafeteria`
Ejercicio 2	`evaluacion01-parqueo`

2 · Estructura de cada repositorio

```
evaluacion01-cafeteria/
├── README.md
├── Producto.java
├── Caja.java
├── Cafeteria.java      ← tiene main
└── capturas/
    └── ejecucion.png
```

```
evaluacion01-parqueo/
├── README.md
├── Vehiculo.java
├── Moto.java
├── Auto.java
├── Camion.java
├── Parqueo.java        ← tiene main
└── capturas/
    └── ejecucion.png
```

No subas los archivos `.class`: son el resultado de compilar, no tu código.

3 · Qué debe tener el README.md

- Tu **nombre completo** y el nombre del curso.
- Una descripción corta de lo que hace el programa.
- Cómo compilarlo y ejecutarlo: `javac *.java` y `java Cafeteria` (o `java Parqueo`).
- Una **captura** del programa funcionando: `![Ejecución](capturas/ejecucion.png)`.

4 · Subir el código con Git

Crea el repositorio vacío en GitHub (botón **New**, visibilidad **Public**, sin README) y desde la carpeta de tu proyecto ejecuta:

```
Terminal · solo la primera vez
git init
git add .
git commit -m "Clase Producto con encapsulamiento"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/evaluacion01-cafeteria.git
git push -u origin main
```

Cada vez que avances, guarda y sube un nuevo commit:

```
Terminal · cada avance
git add .
git commit -m "Menú con do-while y switch"
git push
```

COMMITS

Cada repositorio debe tener **al menos 3 commits** con mensajes que describan el avance. Un solo commit con todo el código al final no cumple este criterio.

5 · Verificar y enviar

1. Abre cada enlace en una **ventana de incógnito**. Si ves el código sin iniciar sesión, el repositorio es público.
2. Envía al docente un solo mensaje con tus datos y los dos enlaces, a más tardar el **lunes 02 de noviembre a las 8:00 p.m.**:

```
Mensaje al docente
Evaluación 01 · Fundamentos de Desarrollo Backend con Java
Nombre: Ana María López Pérez
Ejercicio 1: https://github.com/TU-USUARIO/evaluacion01-cafeteria
Ejercicio 2: https://github.com/TU-USUARIO/evaluacion01-parqueo
```

IMPORTANTE

Se evalúa el **último commit anterior a la fecha límite**. Los cambios que subas después no se toman en cuenta. Un repositorio privado o un enlace que no abre se califica como no entregado.

CAPÍTULO 05

5 RÚBRICA DE EVALUACIÓN

> Cada ejercicio vale **50 puntos**. Estos son los criterios y lo que se revisa en cada uno.

Ejercicio 1 · Cafetería

Criterio	Qué se revisa	Puntos
Funcionamiento	Compila sin errores, el menú funciona y los cálculos del ticket son correctos.	12
Encapsulamiento	Atributos `private`; getters; `setPrecio()` valida; no existe `setStock()` ni setters en `Caja`.	15
Lógica y validaciones	`do-while` y `switch`; rechaza opciones, cantidades y precios inválidos; aviso de stock bajo.	10
Calidad del código	Una clase por archivo, nombres claros (camelCase y PascalCase), indentación y comentarios útiles.	6
Repositorio	Público, con el nombre correcto, README completo con captura y al menos 3 commits.	7
Total		50

Ejercicio 2 · Parqueo

Criterio	Qué se revisa	Puntos
Funcionamiento	Compila sin errores, los cuatro casos de prueba dan el cobro correcto y el reporte final es correcto.	12
Herencia	`extends`, `super(...)` en los constructores, `@Override`, `super.calcularCobro()` en `Camion`; las subclases no repiten atributos.	15
Lógica y validaciones	Reglas de los 15 minutos y de la hora o fracción con `Math.ceil()`; vuelve a pedir tipos y minutos inválidos; acumuladores y placa que pagó más.	10
Calidad del código	Una clase por archivo, atributos `private` en `Vehiculo`, nombres claros, indentación y comentarios útiles.	6
Repositorio	Público, con el nombre correcto, README completo con captura y al menos 3 commits.	7
Total		50

CAPÍTULO 06

6 ANTES DE ENVIAR

Revisa esta lista antes de mandar tus enlaces. Si todas las casillas están marcadas, tu entrega está completa.

CÓDIGO

- [ ] Los dos programas compilan con `javac *.java`.
- [ ] Cada clase está en su propio archivo.
- [ ] Los atributos son `private`.
- [ ] El ticket de 5 sándwiches da C$491.63.
- [ ] Los cuatro casos del parqueo dan el cobro esperado.
- [ ] Probé opciones y cantidades inválidas.

REPOSITORIOS

- [ ] Se llaman `evaluacion01-cafeteria` y `evaluacion01-parqueo`.
- [ ] Son públicos: abren en incógnito.
- [ ] Cada uno tiene README con mi nombre y una captura.
- [ ] Cada uno tiene al menos 3 commits.
- [ ] No subí archivos `.class`.
- [ ] Envié los dos enlaces antes del lunes 02 de noviembre, 8:00 p.m.

¿DUDAS?

Consulta al docente durante las clases del martes 27 y el jueves 29 de octubre. Pregunta temprano: la noche de la entrega no es buen momento para descubrir que Git no está configurado.



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026